Universidade Federal de Sergipe
Campus de São Cristóvão
Departamento de Computação

COMP0472 - Sistemas Operacionais
Prof. Glauco de Figueiredo Carneiro

RELATÓRIO DA ATIVIDADE 3 (ATV3)

Virtualização, Contêineres, Namespaces e cgroups

Álex Felipe Santos Melo
Matrícula: 202200025395

São Cristóvão - SE
10 de novembro de 2026

1-Objetivo

Este relatório documenta o laboratório realizado na Atividade 3 da disciplina COMP0472 - Sistemas Operacionais, cujo objetivo é observar, com comandos e dados do próprio kernel, os mecanismos que isolam processos e limitam recursos em contêineres Linux. O trabalho não reconstrói uma aplicação: ele executa um laboratório pequeno e reprodutível com Docker no notebook do discente.

A pergunta norteadora é: **como namespaces alteram a visão de processos, mounts e rede, e como cgroups limitam recursos sem transformar um contêiner em uma máquina virtual independente?** Especificamente, buscou-se: (1) distinguir VM, hipervisor e contêiner no ambiente efetivamente usado; (2) identificar evidências de namespaces PID, mount e network em dois contêineres; (3) configurar e confirmar quotas de CPU, teto de memória e limite de tarefas, comparando um trabalho fixo sob duas quotas; e (4) demonstrar a negação de escrita em uma montagem somente leitura, relacionando os resultados ao compartilhamento de infraestrutura de inferência.

2-Identificação do Discente

Nome: Álex Felipe Santos Melo

Matrícula: 202200025395

Curso: Engenharia de Computação

Disciplina: COMP0472 - Sistemas Operacionais

Professor: Glauco de Figueiredo Carneiro

Semestre: 2026.2

Data de entrega: 10 de novembro de 2026

Repositório: github.com/alexfsmelo/SO_UFS_2026_2_Melo_Alex_ATV3 (commit da entrega: tag `entrega-atv3`)

3-Materiais e Métodos

Hardware

Todos os experimentos foram executados no notebook do discente, com CPU AMD Ryzen 7 7735HS (8 núcleos e 16 threads) e 16 GB de RAM instalada. O sistema hospedeiro é o Windows 11, que executa o WSL2 sobre o hipervisor Hyper-V. A VM do WSL2 recebeu 4 CPUs lógicas, 7,8 GiB de RAM e 2 GiB de swap.

Software

O Linux do WSL2 é o Ubuntu 26.04 LTS, com kernel 6.6.114.1-microsoft-standard-WSL2. O Docker Engine 29.1.3 roda dentro do próprio WSL2 (pacote do Ubuntu, sem Docker Desktop), com cgroup v2, driver de cgroup systemd e armazenamento overlayfs; o Docker Compose não está instalado e não foi necessário. A imagem do laboratório é a `python:3.12-slim` (Debian

13, Python 3.12.16), fixada por digest. Os scripts são próprios, equivalentes aos do kit de apoio, e estão na pasta `lab/` do repositório.

Metodologia de Medição

Todas as grandezas vêm de arquivos do kernel, nunca de estimativas. Os handles de namespaces foram lidos com `readlink /proc/1/ns/*` dentro dos contêineres e com `/proc/<PID>/status` e `/proc/<PID>/ns/*` no host. Os limites efetivos e os contadores de CPU, memória e tarefas foram lidos pelo próprio processo de carga em `/sys/fs/cgroup` (`cpu.max`, `cpu.stat`, `memory.max`, `memory.swap.max`, `memory.current`, `memory.peak`, `pids.max`), antes e depois do trabalho. O tempo de parede e o tempo de CPU do processo foram medidos com `time.perf_counter()` e `time.process_time()`. Tabelas e gráfico são gerados pelo script `lab/resumo.py` a partir dos registros brutos, e o `lab/validar.py` recalcula as medianas e confere os números deste relatório.

4-Ambiente e Contêiner Reproduzível - Parte A

Ambiente Linux do Daemon

A Tabela 4.1 reúne o registro do ambiente gerado por `lab/ambiente.sh` (arquivo `results/ambiente/ambiente.txt`). O terminal e o daemon do Docker estão no mesmo Linux, de modo que o `/proc` do terminal é o do host do daemon.

Tabela 4.1: Ambiente Linux real do daemon

Item	Valor observado	Evidência
Tipo de ambiente	WSL2 sobre Hyper-V, no Windows 11	`systemd-detect-virt` = wsl; "Hypervisor detected: Microsoft Hyper-V"
Kernel	6.6.114.1-microsoft-standard-WSL2	`uname`; o mesmo valor dentro do contêiner
Distribuição	Ubuntu 26.04 LTS	`/etc/os-release`; PID 1 = systemd
CPUs, RAM e swap	4 CPUs lógicas (AMD Ryzen 7 7735HS with Radeon Graphics), 7,8 GiB, 2 GiB de swap	`nproc` e `/proc/meminfo`
Docker e Compose	Engine 29.1.3, API 1.52; Compose não instalado	`docker version` e `docker compose version`
cgroup	v2, driver systemd	`docker info`; `/sys/fs/cgroup` do tipo cgroup2fs
Controladores	cpuset cpu io memory hugetlb pids rdma	`cgroup.controllers` na raiz
Segurança do daemon	seccomp padrão e cgroup namespace privado	`docker info`, campo SecurityOptions

Fonte: Autor.

VM, Hipervisor e Contêiner

No ambiente usado há uma VM entre o hardware e o kernel Linux: o Hyper-V, hipervisor do Windows, executa a VM leve do WSL2, que tem seu próprio kernel Linux 6.6.114.1-microsoft-standard-WSL2. Essa VM é uma só para todo o ambiente. Os contêineres não ganham VM nem kernel próprios: são processos desse kernel, criados pelo `runc` com namespaces novos e colocados em cgroups próprios. A Figura 4.1 mostra as camadas. O comando `uname -r`

devolve o mesmo valor no terminal e dentro do contêiner, o que confirma o kernel compartilhado.

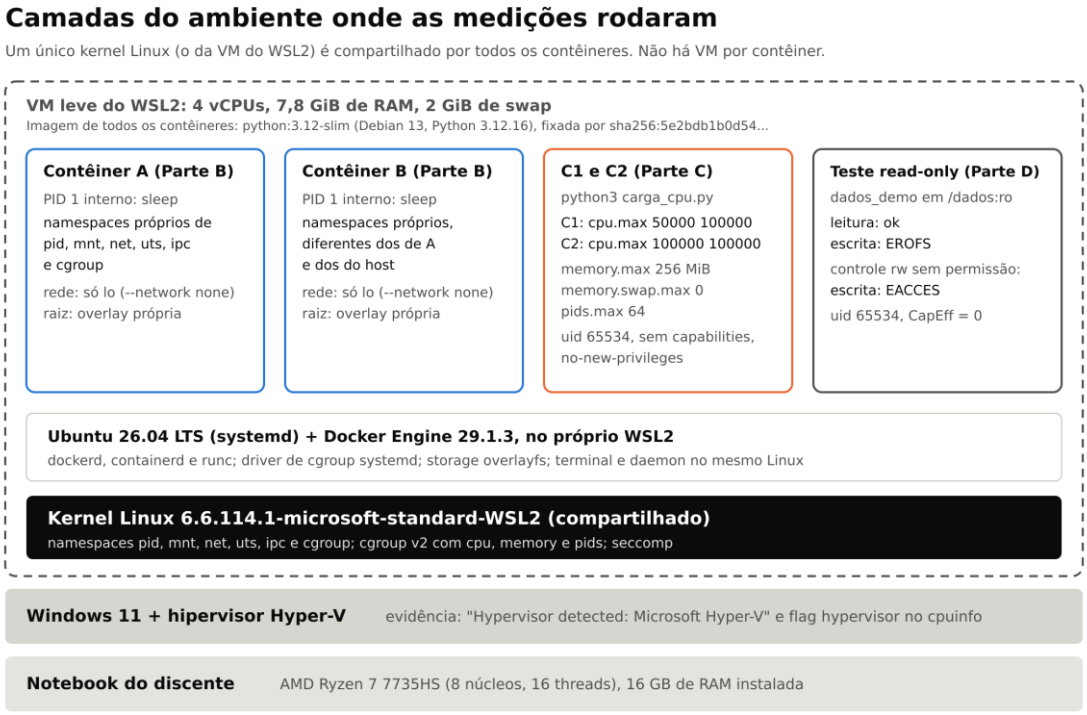


Figura 4.1: Camadas do ambiente: notebook, Windows 11 com Hyper-V, VM do WSL2 com um único kernel Linux, Docker e os contêineres das Partes B, C e D
Fonte: Autor.

Cgroup V2 e Controladores

O WSL2 usa cgroup v2 unificado, que é a versão pressuposta pelo roteiro. Os controladores necessários estão disponíveis e habilitados para os filhos da raiz (`subtree_control`: cpuset cpu io memory pids). Não há limites ancestrais: em `/sys/fs/cgroup/system.slice`, onde o systemd cria os escopos dos contêineres, `cpu.max` vale `max 100000` e `memory.max` e `pids.max` valem `max`. O daemon roda como root, sem modo rootless.

Imagem e Parâmetros Fixados

A imagem foi obtida com `docker pull python:3.12-slim` e fixada pelo digest `python@sha256:5e2bdb1b0d542d5039cffb8181c65930093949194a65c365dfb1b0fe13a4fa5b`, criado em 05/11/2026. Os parâmetros das execuções estão em `results/raw/execucoes/parametros.txt`: N = 40.000.000 iterações, memória de 256 MiB, memória mais swap de 256 MiB, `--pids-limit 64`, usuário 65534, `--cap-drop ALL`, `no-new-privileges` e `--network none`. O código é identificado pelo hash sha256 de cada script, registrado em `ambiente.txt`, e pela tag `entrega-atv3` do repositório.

5-Evidências de Namespaces - Parte B

O script `lab/evidencia_ns_mount.sh` sobe dois contêineres independentes da mesma imagem (`lab3_ns_a_4367` e `lab3_ns_b_4367`), ambos com `sleep 300` como processo

principal e sem rede. Com os dois ativos, coleta o PID informado por `docker inspect`, o comando do PID 1 interno, os handles de `/proc/1/ns/*` e, pelo `/proc` do host, a linha `NSpid` e os mesmos handles. A Tabela 5.1 resume os dois registros reais.

Tabela 5.1: Registros dos dois contêineres

Campo	Contêiner A	Contêiner B
PID no host (`docker inspect`)	4422	4476
PID 1 interno (`/proc/1/comm`)	sleep	sleep
`NSpid` lido no host	4422 1	4476 1
Processos visíveis dentro	PID 1 (sleep) e o próprio `docker exec`	PID 1 (sleep) e o próprio `docker exec`
Interfaces de rede dentro	lo	lo
Raiz do mount namespace	overlay própria, 24 montagens	overlay própria, 24 montagens

Fonte: Autor.

Tabela 5.2: Comparação dos handles de namespaces

Recurso	Handle em A	Handle em B	Iguais?	Interpretação
pid	pid:[4026532297]	pid:[4026532361]	nao	Árvores de PID separadas: cada contêiner numera seus processos a partir de 1 e não vê os do outro nem os do host.
mnt	mnt:[4026532294]	mnt:[4026532358]	nao	Tabelas de montagem separadas: cada um tem sua raiz overlay; montar algo em A não aparece em B.
net	net:[4026532299]	net:[4026532363]	nao	Pilhas de rede separadas: com `--network none`, cada um tem só o seu `lo`; o host tem `eth0` e `docker0`.
uts	uts:[4026532295]	uts:[4026532359]	nao	Hostnames independentes (o ID curto de cada contêiner).
ipc	ipc:[4026532296]	ipc:[4026532360]	nao	Filas, semáforos e memória compartilhada System V separados.
cgroup	cgroup:[4026532298]	cgroup:[4026532362]	nao	Separado: em cgroup v2 o Docker cria um cgroup namespace privado, e o contêiner vê o próprio cgroup como raiz.

Fonte: Autor. O terminal do host tem pid:[4026532219], mnt:[4026532217] e net:[4026531840].

Os handles de PID, mount e rede são diferentes entre A, B e o terminal do host, e o comportamento confirma a separação: cada contêiner vê só o seu PID 1, tem a própria raiz de arquivos e enxerga apenas a interface `lo`. Os handles lidos pelo host em `/proc/4422/ns/*` são idênticos aos lidos dentro de A, o que mostra que se trata do mesmo processo visto de dois lugares.

Como os dois podem ter PID 1. Um namespace de PID é uma tabela de numeração. Quando o `runc` cria o contêiner, o primeiro processo do namespace novo recebe o número 1 nessa tabela e continua tendo um número na tabela do namespace pai [PIDNS]. A linha `NSpid: 4422 1` do `/proc/4422/status` mostra exatamente isso: é a mesma tarefa do kernel, com o número 4422 no host e 1 dentro de A. O contêiner B tem outro processo (4476 no host), que

também é 1 no seu próprio namespace. São dois processos diferentes, com números iguais em tabelas diferentes.

O que os handles não demonstram. Handles diferentes mostram visões separadas, não isolamento completo. Não mostram que o kernel é outro (é o mesmo), nem limite de recursos (papel do cgroup), nem separação de identidade: não foi usado user namespace, então o UID 0 dentro de um contêiner é o UID 0 do host. Também não provam ausência de canais laterais. Além disso, o número do handle é o inode do namespace e só identifica o namespace enquanto ele existe; o kernel pode reutilizá-lo depois.

Tentativa preservada. A primeira coleta foi feita com o usuário comum, que não tem permissão para ler os links `/proc/<PID>/ns/*` de processos do root; o script registrou erroneamente "daemon remoto". O registro foi preservado em `results/namespaces/tentativa1_sem_root/`, o script foi corrigido para informar o motivo real e a coleta foi refeita como root.

6-Configuração do Experimento de cgroups - Parte C

Todas as execuções da Parte C foram feitas na mesma sessão, em 10/11/2026, sem reiniciar o sistema entre as condições.

Carga: `lab/carga_cpu.py` executa N iterações de um gerador congruente linear de 64 bits e acumula cada valor num checksum. O mesmo N sempre gera o mesmo checksum, e pular ou repetir trabalho o altera. Para N = 40.000.000, o checksum `098953c3ffd76aa3` foi recalculado fora do contêiner, sem limites, e coincide com as seis execuções.

Piloto: com N = 2.000.000, C1 levou 1,29 s e C2, 0,61 s. N = 40.000.000 foi escolhido para que C1 durasse cerca de 25 s, abaixo do limite interno de 60 s, e então congelado nas seis execuções.

Ordem: alternada em ABBA (C1 C2, C2 C1, C1 C2), para que uma variação lenta do host não favorecesse sempre a mesma condição.

Confirmação dos limites: a flag pedida não prova o limite. O processo de carga lê o próprio cgroup antes e depois do trabalho e grava os valores efetivos e os deltas de `cpu.stat`; o `docker inspect` fica registrado à parte, como valor pedido.

Segurança: limite interno de 60 s, `timeout 90` no host, contêiner sem rede, usuário 65534, sem capabilities e com `no-new-privileges`.

7-Experimentos Realizados

C1: Quota de 0,5 CPU

`--cpus 0.5`, 256 MiB de memória, memória mais swap de 256 MiB e `--pids-limit 64`, com três repetições. Mede o tempo e a vazão do trabalho fixo quando o cgroup pode usar 50 ms de CPU a cada período de 100 ms.

C2: Quota de 1,0 CPU

`--cpus 1.0`, com os mesmos limites de memória e tarefas, a mesma imagem e o mesmo trabalho, também com três repetições. Mede o efeito de dobrar a quota.

8-Resultados

Tabela Consolidada

A Tabela 8.1 apresenta as seis execuções principais, na ordem real. Todas terminaram com status ok, código de saída 0 e o checksum de referência.

Tabela 8.1: Execuções válidas da Parte C (N = 40.000.000)

Ordem	Condição	Rep.	Parede (s)	CPU do processo (s)	Trabalho (M it/s)	cpu.max	Throttled / períodos	Throttled (s)
1	C1	1	24,792	12,394	1,613	50000 100000	248 / 248	12,390
2	C2	1	12,342	12,334	3,241	100000 100000	6 / 123	0,003
3	C2	2	11,935	11,934	3,352	100000 100000	0 / 119	0,000
4	C1	2	25,069	12,517	1,596	50000 100000	251 / 251	12,544
5	C1	3	25,335	12,682	1,579	50000 100000	253 / 253	12,645
6	C2	3	12,297	12,297	3,253	100000 100000	0 / 123	0,000

Fonte: Autor.

Medianas por Condição

Tabela 8.2: Mediana e variação por condição (n = 3)

Condição	Parede mediana (s)	Trabalho mediano (M it/s)	Mínimo a máximo (M it/s)	Amplitude	CPU efetiva (CPUs)	Throttled mediano (s)
C1 (0,5 CPU)	25,07	1,60	1,58 a 1,61	2,2%	0,500	12,54
C2 (1,0 CPU)	12,30	3,25	3,24 a 3,35	3,4%	1,000	0,00

Fonte: Autor. Amplitude = (máximo - mínimo) / mediana. CPU efetiva = delta de usage_usec / tempo de parede.

A razão entre as medianas de trabalho por segundo foi **2,04** (C2 / C1). Em C1, o cgroup consumiu 0,500 CPU em média e sofreu throttling em todos os períodos de 100 ms. Em C2, consumiu 1,000 CPU; apenas a repetição 1 registrou throttling, em 6 de 123 períodos, somando 3,2 ms, valor desprezível frente aos 12,30 s de execução.

Gráfico Comparativo

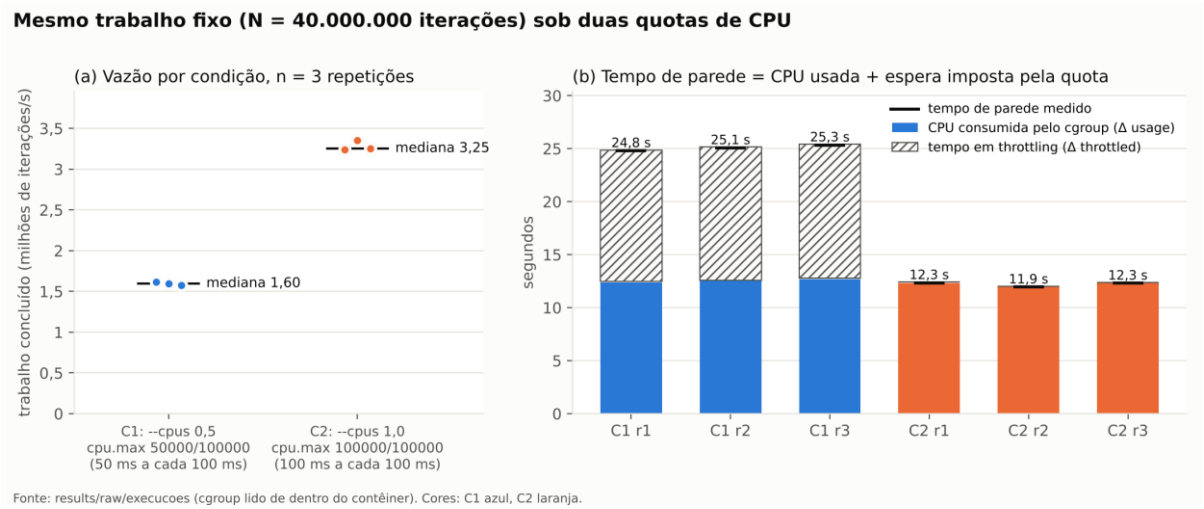


Figura 8.1: Trabalho concluído por segundo e decomposição do tempo de parede em C1 e C2 (n = 3 repetições por condição)

Fonte: Autor.

CPU Usada Contra a Quota

A Figura 8.2 mostra, para cada execução, a CPU efetiva do cgroup (delta de `usage_usec` dividido pelo tempo de parede) contra a quota pedida. As três execuções de C1 ficaram entre 0,499 e 0,501 CPU, e as de C2 entre 1,000 e 1,000: o kernel entregou exatamente o que a quota permite, com diferença na terceira casa decimal.

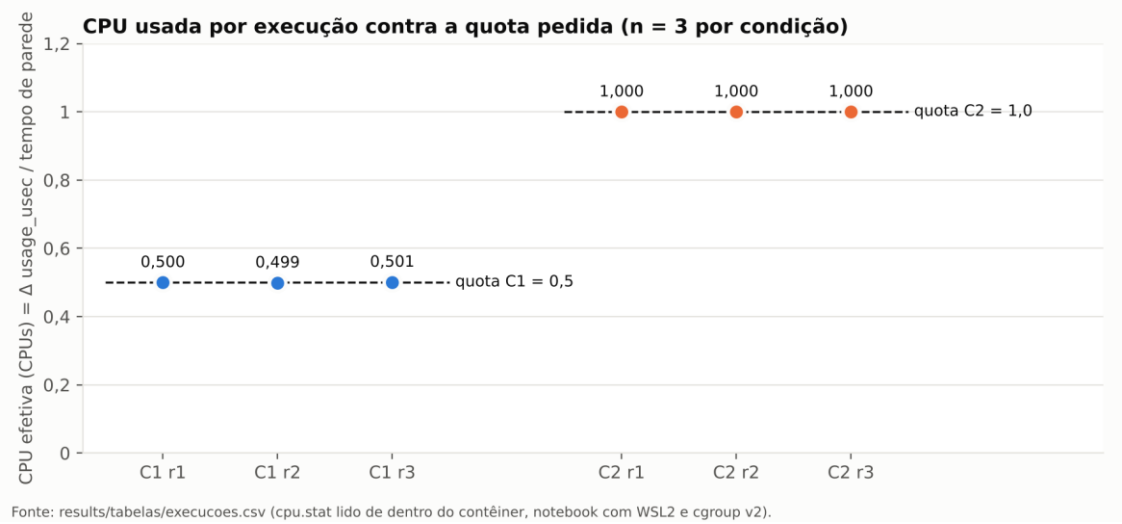


Figura 8.2: CPU efetiva de cada execução contra a quota pedida (n = 3 por condição)

Fonte: Autor.

Memória e Tarefas

Os limites de memória e tarefas foram confirmados em todas as execuções: `memory.max` = 256 MiB, `memory.swap.max` = 0 (o equivalente a memória mais swap de 256 MiB) e `pids.max` = 64, com `pids.current` = 1. A carga é pequena em memória e não se aproximou do teto, e o laboratório não forçou falta de memória. A Tabela 8.3 separa três medidas que não são a mesma coisa.

Tabela 8.3: Medidas de memória por condição (mediana)

Medida	C1	C2	O que representa
RSS do processo	17,0 MiB	17,0 MiB	Páginas residentes do processo, incluindo bibliotecas compartilhadas
memory.current do grupo	11,2 MiB	11,2 MiB	Memória cobrada do cgroup no instante da leitura
memory.peak do grupo	11,2 MiB	11,2 MiB	Maior valor cobrado desde a criação do cgroup

Fonte: Autor.

O RSS é maior que o pico do grupo porque o kernel cobra cada página de cache de arquivo do cgroup que a tocou primeiro [CG]. Parte das páginas do interpretador e das bibliotecas entra no RSS do processo sem ser cobrada do contêiner. Nenhum desses números é "RAM média".

9-Montagem Somente Leitura e Menor Privilégio - Parte D

O diretório sintético `dados_demo/`, com um arquivo `texto.txt` contendo "Documento sintetico", foi montado como `/dados:ro`. Para separar os mecanismos, foram executados três casos, todos sem `--privileged` e sem rede. O teste de escrita usa `open(..., "x")` em Python, que devolve o código de erro exato do kernel. A Tabela 9.1 resume os resultados de `results/mount/readonly.txt`.

Tabela 9.1: Casos de leitura e escrita na montagem

Caso	Identidade	Montagem	Leitura	Escrita	Mecanismo que negou
D1	uid 65534, CapEff 0000000000000000, no_new_privs 1	/dados ro,relatime	ok	negada, errno 30 EROFS (Read-only file system)	Flag read-only da montagem
D2	uid 0 (root), CapEff 00000000a80425fb	/dados ro,relatime	ok	negada, errno 30 EROFS (Read-only file system)	A mesma flag: nem root escreve
D3	uid 65534, CapEff 0000000000000000, no_new_privs 1	/controle rw,relatime	não testada	negada, errno 13 EACCES (Permission denied)	Permissão do diretório (0755, dono 1000)

Fonte: Autor. Depois dos testes, `dados_demo` contém só `texto.txt` e o diretório de controle está vazio.

D1 é o teste pedido: leitura bem-sucedida e escrita negada com **EROFS** (Read-only file system), por um usuário não root, sem nenhuma capability e com `no_new_privs`. D2 mostra que a negação vem da **montagem**, não da identidade: root, com as capabilities padrão do Docker, recebe o mesmo EROFS. D3 mostra outro mecanismo: com a montagem gravável, a escrita é negada com **EACCES** pelas permissões do diretório.

Os conceitos ficam separados assim: o **mount namespace** define quais montagens o contêiner enxerga [MNTNS]; a **montagem read-only** é uma flag dessa montagem, verificada pelo kernel antes das permissões; as **permissões** comparam UID e modo do arquivo; e as **proteções do kernel** (capabilities zeradas, `no_new_privs` e filtro seccomp) reduzem o que o processo pode tentar para contornar as anteriores [CAP]. A negação de escrita não prova isolamento absoluto:

o processo ainda lê tudo o que foi montado, e um defeito no kernel continuaria afetando todos os contêineres.

10-Experimento Extra: Linha do Tempo do Throttling

Este experimento não é exigido pelo enunciado. Foi feito por iniciativa do discente, depois das seis execuções principais, para responder a uma pergunta que elas não respondem e para gerar uma evidência visual do mecanismo de quota.

Por Que Este Teste Foi Feito

As execuções da Parte C leem o `cpu.stat` só no início e no fim do trabalho. Elas provam que a quota foi aplicada e quanto tempo o processo ficou em throttling, mas não mostram como isso acontece ao longo da execução. Duas hipóteses dariam o mesmo total: o kernel poderia deixar o processo mais lento de forma contínua, como se a CPU tivesse metade da velocidade, ou poderia deixá-lo rodar em velocidade plena e pausá-lo quando a quota do período acabasse. A diferença importa para entender a afirmação do enunciado de que a quota é tempo agregado por período, e não reserva de núcleo, e importa na prática: num servidor de inferência, pausas de dezenas de milissegundos aparecem como variação de latência para o usuário. O teste também confere a medição principal por outro caminho, lendo o cgroup pelo host em vez de dentro do contêiner.

Como Foi Feito

O script `lab/linha_do_tempo.py` executou a mesma carga (N = 40.000.000) uma vez em C1 e uma vez em C2, com os mesmos limites e flags da Parte C, e leu o `cpu.stat` do cgroup do contêiner pelo host a cada 20 ms, num total de 1297 leituras em C1 e 738 em C2. O caminho do cgroup vem de `/proc/<PID>/cgroup`, e a leitura pelo host não consome CPU do próprio contêiner. Por ser uma execução única por condição, este teste não entra nas medianas da seção 8.

Resultados

Na Figura 10.1, a CPU acumulada pelo cgroup cresce em linha reta, com inclinação 0,500 em C1 e 1,000 em C2, iguais às quotas; em qualquer janela de 1 s, C1 usou entre 0,49 e 0,51 CPU. Em C1, 267 dos 267 períodos registrados tiveram throttling; em C2, 0 de 151. O detalhe de 1 s de C1 mostra a CPU chegando em degraus, e não de forma contínua.

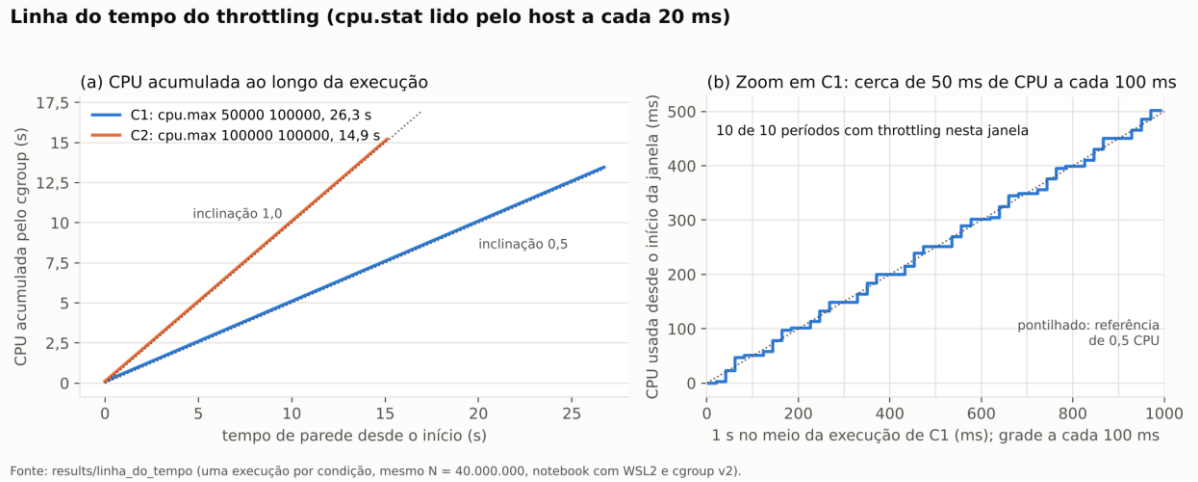


Figura 10.1: CPU acumulada ao longo do tempo em C1 e C2 e detalhe de 1 s de C1 (experimento extra)

Fonte: Autor.

Nesta rodada, o mesmo trabalho gastou mais tempo de CPU que nas execuções principais (13,16 s em C1 e 14,92 s em C2, contra medianas de 12,52 s e 12,30 s), e por isso os tempos de parede foram maiores (26,30 s e 14,93 s). O notebook estava mais lento naquele momento, possivelmente por estado de energia ou temperatura, o que não foi medido. Como a quota é proporcional ao tempo, a inclinação não muda, e as duas execuções terminaram com o checksum de referência conferido.

O Que o Teste Mostra

A quota não deixa a CPU mais lenta: ela liga e desliga o processo. Em média o contêiner de C1 recebe exatamente 0,5 CPU, mas localmente ele roda em velocidade plena até gastar os 50 ms do período e espera o período seguinte [BWC]. Isso confirma que a quota é um orçamento por período, e não um núcleo reservado nem uma CPU mais lenta. Em C2, a inclinação 1,0 sem nenhum throttling confirma que a carga de uma thread nunca passa de uma CPU. Como limitações, a leitura a cada 20 ms não resolve cada período de 100 ms individualmente (os degraus aparecem com 20 a 30 ms), e há uma única execução por condição.

11-Análise dos Resultados

Q1 - Onde está o kernel compartilhado? Há uma VM intermediária?

O kernel compartilhado é o Linux 6.6.114.1-microsoft-standard-WSL2 da VM do WSL2: `uname -r` devolve o mesmo valor no terminal e dentro do contêiner, e os processos dos contêineres aparecem no `/proc` do host com PIDs comuns (4422 e 4476). Existe uma VM intermediária, criada pelo Hyper-V para o WSL2, mas ela é do ambiente e não de cada contêiner: todos os contêineres dividem a mesma VM e o mesmo kernel.

Q2 - Quais diferenças de handles sustentam a separação de PID, mount e network? O que elas não demonstram?

Os handles `pid`, `mnt` e `net` diferem entre A, B e o terminal (Tabela 5.2), e as observações batem com isso: cada contêiner vê só o próprio PID 1, tem raiz overlay própria e apenas a

interface `lo`. Os handles não demonstram kernel separado, limite de recursos, separação de usuários (sem user namespace, root dentro é root fora) nem ausência de canais laterais [NS].

Q3 - Como diferenciar namespace, cgroup, identidade e política de acesso?

- **Namespace** muda o que o processo vê: PIDs, montagens, rede e hostname.
- **cgroup** muda quanto ele pode consumir: CPU por período, memória e número de tarefas [CG].
- **Identidade** é quem ele é para o kernel: UID, GID e capabilities (uid 65534 com CapEff zerado em D1; uid 0 com CapEff 00000000a80425fb em D2).
- **Política de acesso** decide o que ele pode fazer com o que vê: flags de montagem, permissões de arquivo, seccomp e `no_new_privs`. Os casos D1 a D3 mostram três dessas camadas agindo em separado.

Q4 - Como a quota foi confirmada? Houve throttling?

A quota foi lida no cgroup do próprio processo: `cpu.max = 50000 100000` em C1 e `100000 100000` em C2, isto é, 50 ms ou 100 ms de CPU a cada período de 100 ms, somados entre todas as CPUs do grupo. É tempo agregado por período, não afinidade com um núcleo nem reserva [BWC]. Houve throttling em C1 em 100% dos períodos (248 a 253 por execução), com 12,54 s de espera mediana. Os contadores fecham com o tempo e o trabalho: em C1, CPU consumida (12,52 s) mais tempo em throttling (12,54 s) dá aproximadamente o tempo de parede (25,07 s), e a CPU efetiva é 0,500, o valor da quota. Em C2 o throttling foi praticamente nulo porque a carga tem uma única thread e não consegue usar mais de uma CPU: a quota de 1,0 só foi alcançada, ocasionalmente, no limite de alguns períodos. A conclusão vale para as três repetições, não para uma amostra isolada.

Q5 - O que aconteceu com mediana e variação entre C1 e C2? Qual é a principal ameaça à validade?

A mediana de trabalho por segundo foi 1,60 M it/s em C1 e 3,25 M it/s em C2, razão de 2,04. A variação foi pequena nas duas condições (amplitude de 2,2% e 3,4%). A razão ficou muito perto de 2: o mesmo trabalho gastou só 1,8% mais tempo de CPU em C1 (12,52 s contra 12,30 s). Na execução preliminar em nuvem (seção 12.2), essa diferença foi maior e a razão chegou a 2,14, o que sugere que o ruído de um host compartilhado pesa mais que o próprio mecanismo; com três amostras, porém, não é possível afirmar a causa. A principal ameaça à validade é o ruído do host: o notebook executa o Windows e outros processos ao mesmo tempo que a VM do WSL2. Também pesam o n = 3 e a carga de thread única, que torna C2 equivalente a "sem limite" para este programa.

A Figura 11.1 compara os dois ambientes: a vazão absoluta muda com a máquina e com a versão do Python, mas a razão C2 / C1 fica perto de 2 nos dois.

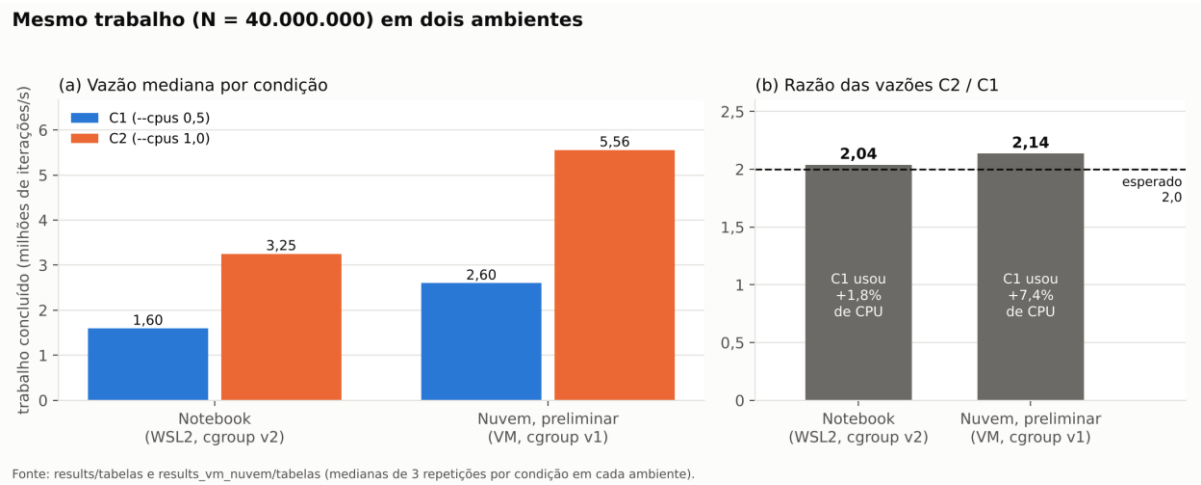


Figura 11.1: Vazão mediana e razão C2 / C1 no notebook e na execução preliminar em nuvem
Fonte: Autor.

Q6 - Por que limitar RAM e tarefas e montar o corpus read-only importa em um servidor de inferência? Que risco permanece?

Num servidor que atende vários usuários ou modelos, o teto de memória impede que uma requisição grande ou um vazamento derrube os vizinhos por falta de RAM; o limite de tarefas impede que um tokenizador, um pool de threads ou um defeito criem processos sem fim; a quota de CPU reparte o processador de forma previsível [RES]; e o corpus do RAG montado como somente leitura impede que um processo comprometido, por exemplo por injeção de prompt que aciona ferramentas, altere os documentos que respondem a todos. Permanecem riscos: o kernel é compartilhado, então uma falha nele atravessa os contêineres; o cgroup de memória não limita VRAM de GPU; a leitura continua permitida, então dados sensíveis montados podem vazar; sem user namespace, root no contêiner é root no host em caso de fuga; e a vazão de disco não foi limitada. O isolamento não dispensa manter kernel e runtime atualizados.

12-Segurança e Limitações

Segurança

- Carga limitada a 60 s dentro do contêiner e a 90 s no host com GNU `timeout`; a execução mais longa durou cerca de 26 s.
- Contêineres sem rede, com usuário 65534, sem capabilities e com `no-new-privileges`, exceto o caso D2, que usa root de propósito para mostrar que a montagem read-only vale também para ele, sem `--privileged`.
- Nenhuma escrita manual em `/sys`, fork bomb, falta de memória deliberada, escalonador de tempo real ou teste fora do próprio notebook.
- Montados apenas `dados_demo/` e um diretório de controle vazio, nunca `/`, a home, o socket do Docker ou pastas sigilosas. Os scripts removem só os contêineres que criaram.
- Registros revisados: `ambiente.txt` usa campos formatados de `docker info`, sem hostname nem usuário, e o registro da Parte D troca o caminho do repositório por `<repo>`.

Limitações e Execução Preliminar

- Três repetições por condição sustentam mediana e amplitude, não inferência estatística.
- A carga é de uma thread; com várias threads, C2 também sofreria throttling e a comparação mudaria.
- O WSL2 limita a VM a 4 CPUs lógicas, das 16 do notebook; os resultados absolutos valem para essa configuração.
- Antes da execução no notebook, o assistente de IA fez uma execução preliminar em uma VM de nuvem (KVM e Firecracker, cgroup v1 e imagem Ubuntu 24.04 com Python 3.12 construída a partir do repositório oficial, porque o Docker Hub estava bloqueado). Os dados estão preservados em `results_vm_nuvem/` e não entram na análise principal. Lá, C1 levou 15,38 s e C2 7,20 s, com razão 2,14: o mesmo comportamento em outro hardware e outra versão de cgroup.

13-Conclusão

Os experimentos respondem à pergunta norteadora. Os namespaces separaram a visão de processos, montagens e rede: dois contêineres da mesma imagem têm handles diferentes, cada um vê apenas o próprio PID 1, e o mesmo processo aparece como 4422 no host e 1 dentro do contêiner. Os cgroups limitaram de fato o consumo: com `cpu.max = 50000 100000`, o contêiner usou 0,500 CPU, sofreu throttling em todos os períodos e levou 25,07 s para o mesmo trabalho que C2 fez em 12,30 s, uma razão de 2,04. Tudo isso aconteceu sobre um único kernel Linux, o da VM do WSL2, sem nenhuma VM por contêiner. O experimento extra acrescentou como esse limite age no tempo: em vez de deixar a CPU mais lenta, o kernel liga e desliga o processo dentro de cada período de 100 ms.

A montagem read-only negou escrita até para root, com um erro diferente do erro de permissão, o que mostra que montagem, identidade e permissões são camadas distintas. Para um servidor de inferência, esses mecanismos permitem dividir uma máquina entre usuários com limites previsíveis e proteger o corpus, mas não substituem a atualização do kernel e do runtime nem resolvem o compartilhamento de GPU.

14-Declaração de Uso de Inteligência Artificial

Declaro que conduzi esta atividade e utilizei a ferramenta de inteligência artificial Claude (Anthropic, identificador `claude-opus-5-5`) como assistente. Decidi executar o laboratório no meu notebook, no WSL2 com Docker, e executei todas as medições analisadas neste relatório: o registro do ambiente, as evidências de namespaces (refazendo a coleta como root para registrar o `NSpid`), o teste read-only, o piloto, as seis execuções C1/C2 e o experimento extra da seção 10, que escolhi para ter mais evidências gráficas. Também defini o formato do relatório e dos slides e sou responsável pela revisão final.

A IA me auxiliou a estruturar a ideia do laboratório a partir do enunciado, a adaptar os scripts da pasta `lab/`, a confirmar a execução de cada etapa (conferindo os registros, o checksum e os limites efetivos) e na redação inicial deste relatório e dos slides. Ela também fez uma execução preliminar em nuvem, preservada em `results_vm_nuvem/` e usada só como conferência.

Nenhuma saída de IA foi apresentada como medição: os números vêm dos arquivos do kernel coletados no meu notebook. Os detalhes, os prompts relevantes e os erros corrigidos estão em `DECLARACAO_IA.md`.

Link do repositório: https://github.com/alexfsmelo/SO_UFS_2026_2_Melo_Alex_ATV3

Link do vídeo: https://youtu.be/NFXl3AMGJl8

15-Referências

[NS] Linux man-pages. namespaces(7). https://man7.org/linux/man-pages/man7/namespaces.7.html

[PIDNS] Linux man-pages. pid_namespaces(7). https://man7.org/linux/man-pages/man7/pid_namespaces.7.html

[MNTNS] Linux man-pages. mount_namespaces(7). https://man7.org/linux/man-pages/man7/mount_namespaces.7.html

[CAP] Linux man-pages. capabilities(7). https://man7.org/linux/man-pages/man7/capabilities.7.html

[CG] Linux Kernel Documentation. Control Group v2. https://docs.kernel.org/admin-guide/cgroup-v2.html

[BWC] Linux Kernel Documentation. CFS Bandwidth Control. https://docs.kernel.org/scheduler/sched-bwc.html

[RES] Docker. Resource constraints. https://docs.docker.com/engine/containers/resource_constraints/